

Received 8 July 2025, accepted 14 August 2025, date of publication 19 August 2025, date of current version 30 September 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3600321

 SURVEY

Effort Estimation in Agile and DevOps: A Systematic Literature Review of Stakeholder Dissatisfaction

SADAQUAT ALI RUK^{1,2}, YASIR MAHMOOD³, AZRI AZMI¹,
NURULHUDA FIRDAUS MOHD AZMI¹, AND SEHAR GUL^{1,4}

¹Faculty of Artificial Intelligence, Universiti Teknologi Malaysia (UTM), Kuala Lumpur 54100, Malaysia

²Department of Computer Science, Shah Abdul Latif University, Khairpur, Ghotki Campus, Khairpur 66020, Pakistan

³Department of Computer Science and Software Engineering, College of Information Technology, United Arab Emirates University, Al Ain, United Arab Emirates

⁴Department of Computer Science, Sukkur IBA University, Sukkur 65200, Pakistan

Corresponding authors: Yasir Mahmood (yasir.mahmood@uaeu.ac.ae) and Sadaquat Ali Ruk (ruk20@graduate.utm.my)

This research is fully funded by United Arab Emirates University (UAEU) under the Start-Up Research Grant with fund No 12T051 and Grant No. G00004635.

ABSTRACT Effort estimation is a critical aspect of software project management, directly influencing the budget, schedule, and resource allocation throughout the software development life cycle. Accurate estimation plays a central role in aligning team capabilities with stakeholder expectations, ensuring timely delivery and efficient use of resources. Despite the adoption of Agile and DevOps methodologies, which emphasize flexibility, rapid iterations and continuous integration, estimation inaccuracies remain a problematic area. These inaccuracies often result in project delays, cost overruns, and scope creep, ultimately leading to stakeholder dissatisfaction and a breakdown of trust in the development process. To investigate these challenges, a Systematic Literature Review (SLR) was conducted, surrounding 1,088 publications retrieved from six major digital libraries: Scopus (201), ScienceDirect (155), Springer (212), Google Scholar (145), IEEE (185), and ACM (190). This research examines the main drivers of estimation challenges in Agile and DevOps environments. Key findings indicate that human-centric issues (cognitive biases, lack of experience), technical issues (fluctuating requirements, lack of adequate history), and process inefficiencies (communication breakdown, poor planning) exert considerable influences on estimation accuracy. To address these challenges, the study categorizes and analyzes a range of estimation techniques, including machine learning models, expert-calibrated hybrid approaches, analogical estimation methods, and process calibration practices such as retrospectives. Findings reveal that a combination of data-driven tools and stakeholder-aligned strategies offers the most promise in narrowing the expectation experience gap. This research provides practitioners with practical recommendations and lays the ground for further study in developing estimation techniques to mitigate the expectation-experience gap.

INDEX TERMS Agile software development, DevOps, estimation methods, estimation accuracy, forecasting, system development life cycle.

I. INTRODUCTION

Digital transformation in the modern era presents enterprises with extraordinary challenges and unique hurdles to overcome [1]. The increasing demand for customized software applications and the ever-increasing need for getting them

to the market are becoming crucial to sustain a competitive edge [2]. The earlier software development models such as Spiral Model, Waterfall Model, Iterative Model, etc. have quite frequently been characterized by relatively low software development success rates. Sometimes these methodologies are described as rigid and slow, which reduces their agility and responsiveness to changes in a dynamic business environment [3].

The associate editor coordinating the review of this manuscript and approving it for publication was Asadullah Shaikh¹.

These Models have successfully been replaced by Agile Methodologies such as Scrum and Kanban, which emphasize on flexibility, iterative development, and customer involvement [4]. Software development teams have constantly struggled to give accurate effort estimates to stakeholders in a lot of cases especially in the context of agile projects [5], [6]. But the industry has recently shown DevOps, an advanced methodology for software development that places an emphasis on agility with a strong focus on operational efficiency and continuous delivery [7]. DevOps provides a primary guarantee of fast and efficient product delivery through continuous releases during the development process [8]. Agile DevOps extends its collaboration by including Agile methodologies into the DevOps model. Within this paradigm, software effort estimation becomes a very crucial aspect.

In the current literature, various studies on DevOps explore methods for aligning development and operations teams [9], [10], the organizational benefits derived from this integration, and the challenges that can emerge during its adoption. However, while the combination of Agile and DevOps has been widely studied in terms of team alignment and organizational benefits, their integration with respect to effort estimation remains underexplored. Despite significant advancements in effort estimation techniques, accurately predicting software development effort continues to pose challenges, particularly within the area of Agile and DevOps environment.

This study is part of the larger research on developing improved effort estimation models for Agile and DevOps-based projects. A first step towards developing such models is to identify the reasons for stakeholder dissatisfaction with current estimation practices and the factors that affect estimation accuracy. Therefore, this paper aims to present a systematic analysis of these factors and propose best practices and recommendations to enhance estimation accuracy and transparency. The target audience for this review is twofold: First, we aim at researchers who seek a structured overview of the challenges and influencing factors in Agile and DevOps effort estimation. Second, we aim at practitioners who wish to discover actionable strategies that support systematic managerial decision-making, build stakeholder trust, and promote processes that reconcile the tension between stability and evolutionary improvement in software evolution.

A. BACKGROUND

Effort estimation has always been among the crucial management terms in software projects which contribute significantly to planning, budgeting, and resource allocation, thus cutting through the successfulness of the project [11], [12]. Estimation refers to the expected spending of resources needed to achieve a project. It necessarily takes into account two important aspects, cost and effort. Cost signifies the value assigned in monetary (\$) terms for the completion of a project, while effort refers to the time requirement for undertaking a project. In the software industry, the ability to accurately estimate effort is crucial for managers to

predict timelines, allocate budgets, and determine the workforce needed to successfully complete a project [12], [13]. On the other hand, inaccurate estimates, insufficient budgets, or flawed forecasts can lead to project failure, resulting in significant financial losses and a decline in sales [14]. Accurate effort estimation is crucial in determining the outcome of software projects, significantly impacting their success or failure [15]. According to research published in the United States by the International Function Point Users Group [16], there's a clear need to improve agile effort estimation technique. According to book by [17] has shown that as much as 65% of Agile projects fail, largely because of the dynamic nature of Agile, which can make the clarity and stability of project requirements harder to sustain throughout the development cycle. Hence, the question of having a stable foundation in the early stages of development but at the same time obtaining iterative feedback is a very challenging one. Incorporating DevOps alongside Agile offers a potential solution to this challenge.

B. EFFORT ESTIMATION IN DEVOPS

The DevOps movement emerged between 2007 and 2008 in response to major inefficiencies in the IT industry, especially the disconnect between software development and IT operations teams [18] presented the talk on “10 deploys per day; Dev & ops cooperation at Flickr” at Velocity Conference. They discussed how Flickr achieved rapid deployments through the combination of Dev and Ops. This talk significantly influenced the adoption of DevOps practices in modern software development challenging the traditional waterfall approach and sparking widespread interest in DevOps culture.

This transformation not only reshaped deployment strategies but also raised new challenges in effort estimation [19]. Unlike traditional models that assume stable requirements and sequential development, the DevOps paradigm introduces rapid releases, frequent code changes, automated testing, and infrastructure as a code, all of which complicate the prediction of effort and resource needs. Research indicates that the estimation techniques applicable in traditional software development are insufficient for the agile and collaborative environment promoted by DevOps. For instance, [20] highlighted that the increasing complexity of modern software projects necessitates more sophisticated estimation standards and techniques, especially as agile methodologies continue to evolve into DevOps-driven delivery models. Consequently, there is a growing need for adaptive, real-time estimation approaches that can leverage historical pipeline data and accommodate the continuous, automated nature of DevOps environments.

II. RESEARCH QUESTION

We structured our research questions using the PICOS framework (Population, Intervention, Comparison, Outcomes, Study Design) as outlined by [21]. This approach is

TABLE 1. Application of PICOS Criteria in the Study.

Parameter	Meaning	Terminologies
P (Population)	Can be a specific role, an application area, or an industry domain.	Agile and DevOps practitioners, software development teams, and relevant studies in software engineering.
I (Intervention)	The methodology, tool, or technology that addresses a specific issue.	Effort estimation techniques, including ensemble models integrating algorithmic, non-algorithmic, and machine learning approaches.
C (Comparison)	The methodology, tool, or technology in which the intervention is being compared.	Comparison of traditional estimation methods vs new development methodologies.
O (Outcomes)	Factors of importance to practitioners and/or the results that intervention could produce.	Identification of key factors causing stakeholder dissatisfaction with effort estimation (RQ1) and key approaches to improve effort estimation in agile devops projects (RQ2)
Context	The context in which the comparison takes place.	Agile Devops Development Environment.

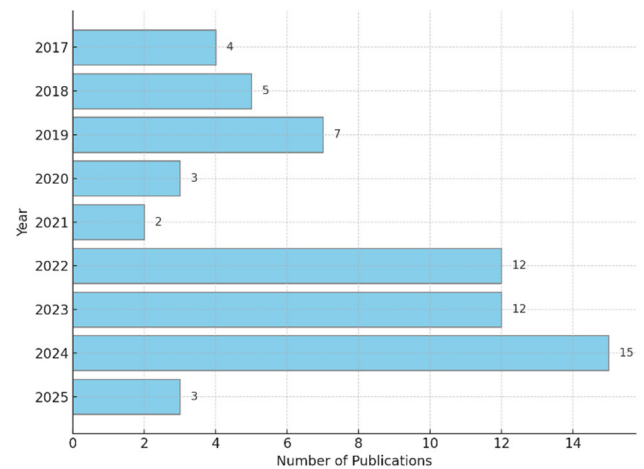
commonly used to structure research questions for systematic literature reviews. Table 1 presents how each component of the PICOS framework was adapted for the context of this study.

Building upon the PICOS framework, we formulated two research questions (RQs) to guide our systematic literature review. These questions aim to address the challenges and opportunities in effort estimation within Agile and DevOps environments, focusing on both the root causes of stakeholder dissatisfaction and potential solutions for improvement.

RQ1: What are the key factors contributing to stakeholder dissatisfaction with effort estimation in Agile and DevOps projects?

RQ2: What are the most effective approaches or techniques for improving effort estimation accuracy and reliability in Agile and DevOps development environments?

To maintain the rigor and consistency of our research, we conducted this systematic literature review (SLR) in accordance with established methodologies tailored for secondary studies in software engineering. Specifically, we adhered to the Software Engineering Guidelines for Reporting Secondary Studies (SEGRESS) framework, as proposed by [22]. It offers better adaptations in the context of software engineering research [11]. Moreover, our approach aligns with established best practices in empirical software engineering, highlighting the importance of a

**FIGURE 1.** Number of publications per year related to effort estimation in Agile and DevOps projects (2017–2025).

methodical search strategy that directly supports the integrity and reproducibility of the evidence collected, followed the recommendations outlined by [23], which emphasize minimizing bias and enhancing transparency. In the following sections, we elaborate on our approach to designing the search strategy, selecting studies, and analyzing the collected data.

A. SEARCH STRATEGY

To identify relevant studies, we developed a comprehensive query string based on the Population, Intervention, Comparison, Outcome, and Context (PICOC) criteria. This query was applied across six prominent digital libraries:

- IEEE Explore,
- ACM Digital library
- Scopus
- Google Scholar
- Science Direct
- Springer

These databases were selected for their broad coverage of peer-reviewed literature in engineering, computer science, and interdisciplinary domains, ensuring a robust and diverse collection of studies. The search strategy was applied to publications from 2017 to 2025, using a combination of keywords, boolean operators, and filters such as publication year, document type, and subject area to maximize the retrieval of relevant literature and minimize bias [24]. Figure 2 illustrates the complete search strategy used in this study.

B. QUERY STRING

A query string based on the PICOS framework is developed to address research questions. The search method effectively identified studies focused on Agile-DevOps environments, exploring effort estimation techniques and drawing comparisons between traditional and modern methodologies. This was achieved through an ecosystem-based approach that

also highlighted stakeholder dissatisfaction and proposed strategies for improvement. The main keywords alongside their synonymous search terms appear in Table 2. We have identified essential search terms for each PICOC criterion according to our PICOC framework. We built the query string using boolean operators (AND, OR) combined with truncation to cover various terminology forms and obtain extensive literature coverage. This general query string acts as a flexible and adaptable template which can be customized to match the unique syntax requirements of different digital libraries such as IEEE explore, ACM digital library, Science direct, Scopus, Springer & Google scholar.

("software teams" OR "software engineering" OR "empirical studies" OR "case studies" OR "DevOps practices" OR "agile practices") AND ("effort estimation" OR "software estimation" OR "predict* effort" OR "resource estimation" OR "work effort" OR "estimation accuracy" OR "data-driven estimation" OR "predictive model*") AND ("machine learning" OR "artificial intelligence" OR "deep learning" OR "neural network*" OR "ensemble model*" OR "regression model*" OR "traditional estimation methods" OR "new development methodologies" OR "benchmarking") AND ("issues" OR "estimation issues" OR "estimation difficulties" OR "estimation limitations" OR "estimation inefficiencies" OR "key factors" OR "stakeholder communication" OR "stakeholder concerns") AND ("agile" OR "devOps" OR "agile Development" OR "methodology" OR "continuous Integration" OR "automation" OR "uncertainty estimation") AND pubyear >= 2017 AND pubyear <= 2025.

III. STUDY SELECTION AND QUALITY ASSESMENT

This study maintained high-quality standards while ensuring alignment with research questions (RQs) by applying 18 inclusion, exclusion, and quality criteria to manually extracted key information from each paper throughout multiple phases. Similar to prior studies, we conducted a multi-phase study selection process. As illustrated in Figure 2, we initially collected 1,088 studies. We removed 48 duplicate papers from our sources, leaving 1,040 papers for screening. During the initial screening stage based on titles and abstracts, we excluded a further 241 papers, resulting in 799 papers proceeding to Stage 1. Afterwards, inclusion and exclusion criteria were applied to the title, abstract, and metadata of each study (Stage 1). The research by [25], the Cohen's Kappa statistic was applied to maintain a strict unbiased method for determining agreement between two researchers. The iterative process made adjustments continuously to the IC/EC until a Kappa value of over 0.8 was attained ($\kappa > 0.8$), which demonstrated strong agreement between them. This process reduces selection bias and ensures consistent application of the criteria. As a result of Stage 1 screening, 399 studies were excluded, leaving us with 400 studies. Next, we applied the exclusion/inclusive criteria to the full text (Stage 2) to remove studies that did not fit our scope. Moving forward to quality assessment in Stage 3.

Finally, we assessed the quality of the remaining studies using predefined quality criteria (Stage 3). During these two stages, 340 studies were excluded for not aligning with the scope and quality standards, leaving a total of 60 studies. After Stage 3, we conducted backward and forward snowballing, which led to the inclusion of 3 additional relevant studies, bringing the final total to 63. The first and second author manually conducted these processes without relying on automated review tools. Any disagreements in study selection were discussed between the first and second authors until a consensus was reached.

A. INCLUSION CRITERIA (IC)

IC-1: The studies should be published and also be peer reviewed for journals, workshops, and conferences.

IC-2: The studies should analyze Agile, DevOps, and Agile - DevOps methodologies.

IC-3: The studies should analyze techniques of effort estimation in Agile, DevOps, Agile-DevOps environments.

IC-4: The studies should analyze ensemble models of effort estimation for algorithmic, non-algorithmic, or machine learning approaches.

IC-5: The studies must include case studies, experiments, or surveys to report an outcome.

IC-6: The studies must be written in English to ensure consistency in analysis and interpretation.

B. EXCLUSION CRITERIA (EC)

EC-1: Studies without full-text access are excluded.

EC-2: Duplicate or previously included studies are excluded.

EC-3: Studies focusing only on effort estimation in software maintenance, testing, or effort distribution are excluded.

EC-4: Studies addressing project management topics (e.g., staff allocation, scheduling) instead of effort estimation are excluded.

EC-5: Literature reviews, experience reports, or replication studies without new empirical data are excluded.

EC-6: Studies lacking methodological rigor or empirical validation are excluded.

C. QUALITY CRITERIA (QC)

QC-1: Clearly defines research objectives and questions.

QC-2: Provides a detailed effort estimation methodology.

QC-3: Includes empirical validation (case studies, experiments, or surveys).

QC-4: Reports key performance metrics (e.g., MAE, RMSE, MMRE).

QC-5: Published in a reputable peer-reviewed journal or conference.

QC-6: Discusses study limitations and validity threats.

IV. RESULTS AND DISCUSSION

This section presents insights derived from the review of 63 carefully selected research articles, forming the backbone of our investigation into estimation challenges within

TABLE 2. List of keywords and synonyms.

Keyword	Synonymous Terms
Software teams	Software engineering, Empirical studies, Case studies, DevOps practices, Agile practices
Effort estimation	Software estimation, Predict* effort, Resource estimation, Work effort, Estimation accuracy, Data-driven estimation, Predictive model*
Machine learning	Machine learning, Artificial intelligence, Deep learning, Neural network*, Ensemble model*, Regression model*,New development methodologies, Benchmarking
Issues	Estimation issues, Estimation difficulties, Estimation limitations, Estimation inefficiencies, Key factors, Stakeholder communication, Stakeholder concerns
Agile	DevOps, Agile Development, Methodology, Continuous Integration, Automation, Uncertainty estimation

Agile and DevOps environments. Figure 1 illustrates the annual distribution of these studies from 2017 through mid-2025: publications rose from four in 2017 to seven in 2019, dipped to three and two in 2020–21, likely reflecting pandemic-related slowdowns and then increased sharply to twelve each in 2022 and 2023, peaking at fifteen in 2024. The first half of 2025 already accounts for three papers, indicating that the field’s momentum remains strong. The objective is to extract meaningful patterns and findings that inform our understanding of the persistent gap between effort estimation and stakeholder expectations, despite the adoption of iterative, flexible methodologies.

A clear classification emerges from the analyzed dataset, as shown in Figure 3. The articles are categorized into three main types: journal articles, conference papers and book chapters. Of the total, 48 (76.1%) are journal publications, while 9 (14.2%) are conference papers and 6 (9.5%) are book chapters.

This skew toward journals reflects a strong academic engagement with the nuanced, often complex nature of software estimation in modern development practices. Further analysis of the source databases, as detailed in Table 3, shows that the majority of the reviewed papers were retrieved from Scopus (17), Google Scholar (14), Springer (11), IEEE (8), Elsevier (8), and ACM (4). This diverse range of sources demonstrates comprehensive coverage of both academic and grey literature. Additionally, Figure 4 illustrates the thematic orientation of the studies, with 34(54%) papers focusing solely on Agile, 20(31.7%) on DevOps, and 9(14.3%) addressing both. This distribution indicates that while Agile-centric estimation issues dominate scholarly discussions, there is a growing interest in integrated Agile DevOps perspectives.

To further understand the knowledge structure, a word map was generated based on the most frequently occurring terms in the corpus (see Figure 5). This visualization highlights dominant keywords such as software engineering, effort estimation, project planning, software metrics, estimation accuracy, devops indicating the central focus areas of contemporary research. The prominence of terms like machine learning, agile methodologies, and resource allocation suggests a growing interest in intelligent, adaptive estimation frameworks.

In addition, a keyword co-occurrence network was constructed to analyse how research themes are interlinked (see

TABLE 3. Database Wise Distribution of Papers.

Sources	Papers Count
Scopus	17
Springer	11
Elsevier	8
IEEE	8
Google Scholar	14
ACM	4
Total	63

Figure 6). The resulting clusters indicate three major research streams:

- Cluster 1(Green): Metrics and Performance-Driven Estimation: This cluster includes keywords such as estimation techniques, performance metrics, estimation accuracy, resource allocation, and development life-cycle, suggesting a focus on systematic evaluation, metric-based estimation models, and strategic resource planning in software engineering.
- Cluster 2 (Blue) – Agile Project Management and Engineering Foundations:With terms like project management, software development, software engineering, and agile practices and devops. This cluster captures research concerned with foundational engineering practices and integration into agile management structures.
- Cluster 3 (Red) – Effort Estimation Challenges and Agile Implementation: This cluster prominently features effort estimation, project planning, agile software development, and story points, emphasizing the practical and methodological difficulties in accurately estimating efforts within agile and DevOps settings.

Table 4 presents the interconnected themes visually reinforce the four core categories of stakeholder dissatisfaction identified in the literature: (1) Human-Centric Factors, (2) Technical Constraints, (3) Process Gaps, and (4) Communication and Transparency Issues. These categories were systematically derived through a structured synthesis of existing studies and are further supported by the keyword co-occurrence clusters identified in this review. Specifically, the clusters highlight how technical constraints and process gaps are reflected in metrics- and performance-driven estimation research, while human-centric factors and communication and transparency issues emerge from agile project management and effort estimation challenges. These themes

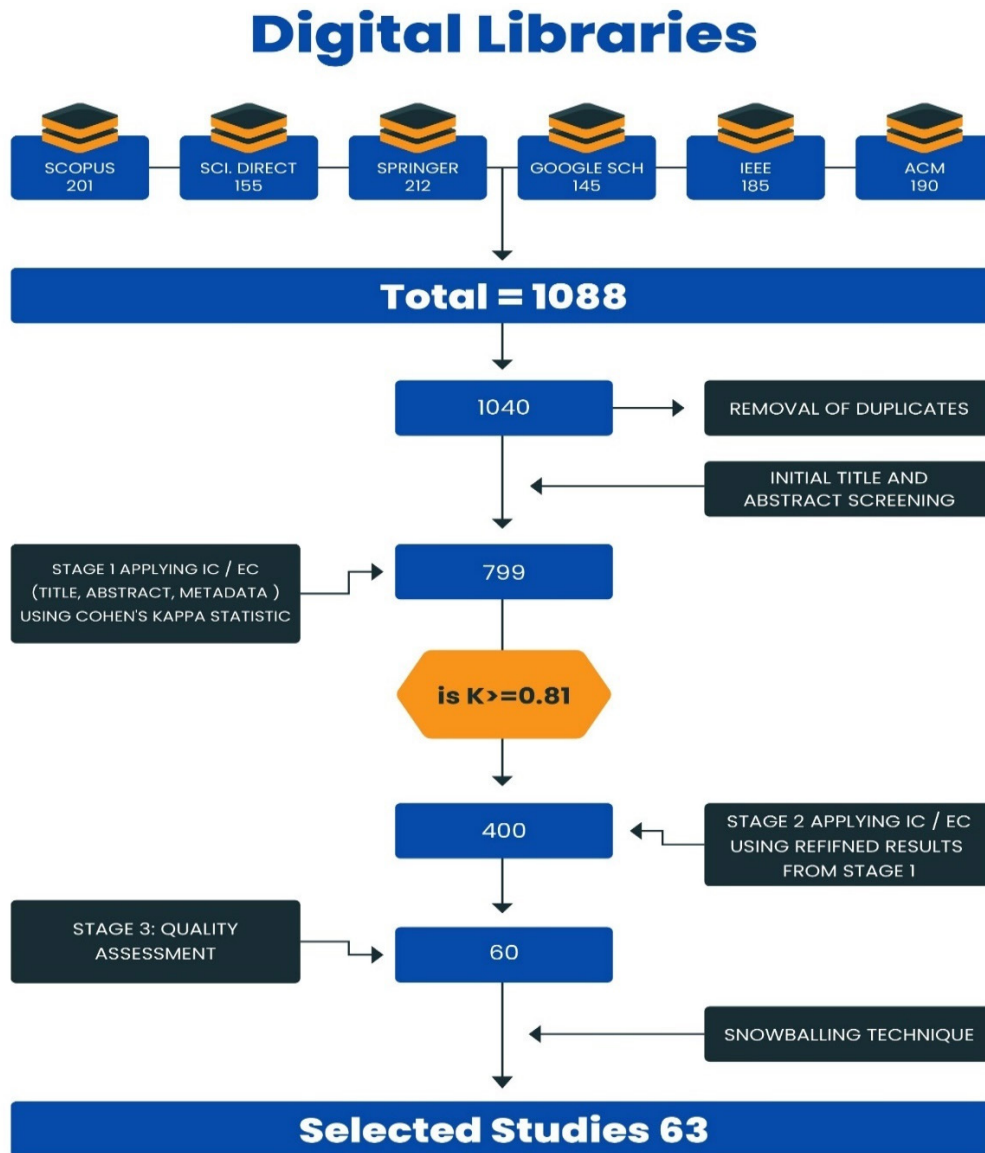


FIGURE 2. Applying PRISMA-based flow diagram of study selection.

encapsulate a range of specific issues that collectively impact the accuracy, trustworthiness, and stakeholder perception of estimation practices in Agile and DevOps settings.

A. HUMAN-CENTRIC FACTORS

Human-Centric Factors were identified in 12 out of 63 studies. The most commonly cited issues included cognitive biases such as over-optimism and anchoring, as well as inexperience with Agile or DevOps-specific estimation methods. In [S3], the authors specifically note that in Agile development, effort estimations are often conducted by teams with varied levels of experience, and many practitioners lack the necessary training in estimation techniques. This results in inaccurate story point assessments, especially when new teams attempt to forecast effort without a shared baseline or adequate exposure to similar past projects. Furthermore, only

a subset of team members typically participates actively in estimation sessions, while others either passively agree or rely on more dominant voices. This imbalance introduces consensus bias and can distort the team's collective understanding of work scope and complexity. Similarly, [S8] identifies human factors such as coordination difficulties, team size imbalance, and developers' confidence in providing story point estimates as major contributors to estimation inaccuracy. The study underscores how anchoring bias can emerge during group estimation sessions like Planning Poker, where initial estimates influence the final decision regardless of rational justification.

Developers sometimes select easier user stories first during sprints, unintentionally accepting complex stories and increasing project risk. The lack of standardization in how expertise is used and the variation in how team members

TABLE 4. Stakeholder dissatisfaction factors from primary studies (N=63).

Categories	Study ID	Count	Percentage
Human-Centric Factors			
- Cognitive biases (over-optimism, anchoring). - Lack of experience in Agile/DevOps. - Resistance to estimation techniques - Misalignment in effort interpretation between teams.	[S1][S2][S3][S8][S11][S12][S13][S33][S34][S35][S54][S57]	12	19.04%
Technical Constraints			
- Lack of historical data for estimation models. - Fluctuating or poorly defined requirements. - Tool fragmentation across CI/CD. - Inability to integrate real-time metrics into planning.	[S2][S3][S5][S6][S7][S8] [S9][S10] [S14][S15][S16][S17][S18][S19][S20][S21][S22][S23][S25][S28][S29][S30][S31][S36][S37][S39][S40][S42][S44][S47][S48][S50][S52][S55][S57][S58][S59][S60]	39	61.90%
Process Gaps			
- Absence of estimate calibration mechanisms. - Lack of feedback loops post-deployment. - Sprint planning disconnected from DevOps workflows - Misuse of velocity or story points	[S1][S3][S4][S6][S7][S8][S9][S10] [S11][S12][S13][S14][S15][S16][S17][S18][S19][S20][S21][S23][S24][S25][S26][S27][S28][S29][S31][S33][S34][S35][S36][S37][S38][S39][S40][S41][S43][S45][S46][S49][S51][S53][S56][S58][S61][S62]	44	69.84%
Communication and Transparency Issues			
- Inadequate stakeholder involvement in estimation. - Poor visibility into estimation logic and assumptions. - Lack of explanation for delays. - Fragmented updates during iteration cycles	[S4][S11][S24][S26][S27][S33][S38][S41] [S43][S63]	10	15.87%

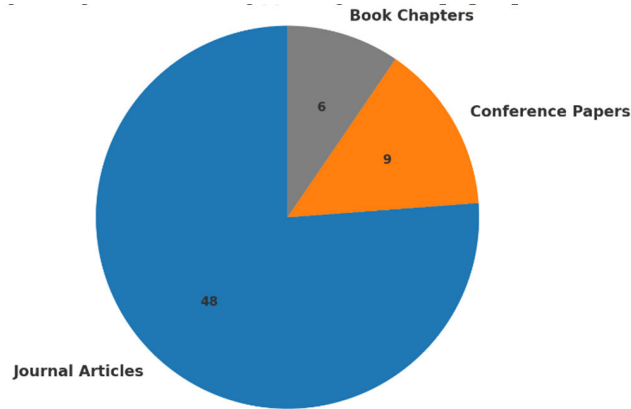


FIGURE 3. Categories of selected studies.

interpret effort further complicates accurate forecasting, leading to stakeholder dissatisfaction when expectations aren't met. Study [S13] further investigates human variability in the application of Agile estimation methods like Planning Poker, Affinity Estimation, and the Bucket System. While accuracy between methods was not significantly different, the time and effort to use each varied. The results revealed that less experienced participants often struggled with method efficiency, and estimation outcomes were heavily influenced by the level of training and familiarity with the technique. The researchers argue that in Agile environments, where teams are expected to produce frequent and accurate estimates, inconsistency in

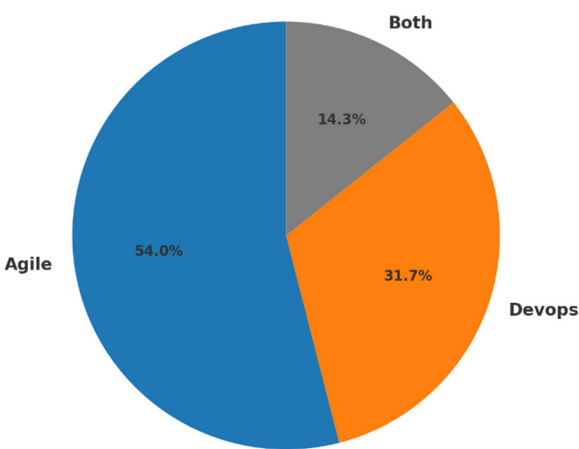


FIGURE 4. Publication distribution by methodology (Agile, DevOps, Both).

skill levels across team members can lead to misalignment in sprint planning and stakeholder expectations. Study [S34] reinforces the importance of practitioner expertise. Through practitioner interviews, it identifies developer experience, specifically with estimating and implementing similar backlog items, as one of the most critical factors in estimation accuracy. However, interestingly, developers did not perceive prior estimates for similar tasks as directly helpful, suggesting a lack of standardized reflection or feedback mechanisms. This gap may be caused by cultural or process issues, where



FIGURE 5. Word map depicting the most frequently used terms in the reviewed literature corpus.

past experiences are not used properly, often because of time pressure or not seeing the value in looking at what was done before.

These findings collectively emphasize that stakeholder dissatisfaction often originates not just from estimation inaccuracies themselves but from the inconsistent and unstructured ways human teams engage in estimation. Variability in experience, training, and judgment, compounded by insufficient calibration or retrospective evaluation, create a fragile estimation culture that can derail Agile and DevOps momentum.

B. TECHNICAL CONSTRAINTS

Technical Constraints were addressed in over half of the reviewed studies (e.g., [S2], [S3], [S5], [S6], [S15], [S39], [S60]). A common theme across various studies is the unavailability of usable historical data for model training and benchmarking; thereby, the reliability of predictive estimation techniques is impaired. For example, the study [S39] [S57] emphasizes that novice Agile teams often struggle with tool fragmentation in CI/CD pipelines, leading to inconsistent data collection and integration, which hinders accurate planning and estimation. Similarly, [S3] and [S17] describe how vague or ambiguous requirements, particularly in the early stages of project initiation, introduce variability and complexities into the estimation process, further exacerbating the technical restrictions. Another technical limitation is indicated by [S2], which points out that mobile application testing, especially in environments with various device and network conditions, introduces unique estimation challenges. According to presented COSMIC-based regression model which tries to address this by incorporating non-functional and test characteristics, yet highlights that such domain-specific variations still complicate technical forecasting. A regression-based solution is offered by [S5] for Agile testing in small web projects but draws attention to the data normalization and transformation complications that stem from mapping non-Gaussian data sets; again, contextualizing limitations in standardized technical estimation processes.

Additionally, [S15] criticizes the popular Planning Poker approach for its weakness to human estimation favoritisms, proposing an AI-assisted alternative leveraging historical backlog data. However, the success of this model is contingent on the availability and quality of large, labeled datasets, an implicit technical constraint in environments lacking mature data infrastructures. In a similar study, [S6] tackle the parameter optimization of estimation formulas using swarm intelligence but highlight that variability in team velocity and user story granularity affects prediction reliability, particularly in the absence of refined historical records. [S57] touches on the limitations in real-time integration, noting that Slack is used post-factum for notifications. While real-time updates exist, they are not deeply embedded into predictive planning models or proactive estimation mechanisms.

C. PROCESS GAPS

Process-related deficiencies were the most common, cited in over 40 studies. Gaps included misalignment between sprint planning and DevOps workflows, lack of calibration loops, and misuse of velocity metrics. Many Agile teams continue to plan iterations using isolated tools or spreadsheets, disconnected from the operational realities managed through CI/CD pipelines. This disconnect often leads to estimates that are outdated or invalidated by rapid downstream changes, such as deployment issues or test failures. For instance, [S26] highlight how fragmented toolchains in DevOps environments prevent seamless integration between planning and execution, causing delays and estimation drift across sprints.

Moreover, estimation practices frequently lack formal feedback or calibration mechanisms. Once a sprint concludes, teams rarely revisit original effort assumptions to assess their accuracy or refine future estimates. Without retrospective analysis or model tuning, the same errors tend to recur across sprints and projects. [S34] identify this challenge and propose a Quality Improvement Paradigm (QIP)-based framework to integrate feedback into estimation cycles, improving both learning and future forecast reliability. Similarly, [S58] argue that popular estimation models such as COCOMO I are inadequate for Agile, lacking real-time adjustment mechanisms. Their proposed predictive method introduces contextual calibration based on user story complexity and developer experience.

Another persistent challenge is the overreliance on velocity and story points as proxies for effort. While these metrics are intended to represent relative complexity and team throughput, they are often misunderstood or manipulated to meet stakeholder expectations. [S54] [S14] found that human-estimated story points poorly correlate with actual development effort in over 90% of the projects analyzed, and were inconsistent across iterations in 78% of cases. [S16] further note that velocity-based estimation is often applied without accounting for variation in sprint content or team capacity, leading to misleading forecasts. [S6] attempted to mitigate this issue through a hybrid optimization model based on velocity and story points, yet even they acknowledge the

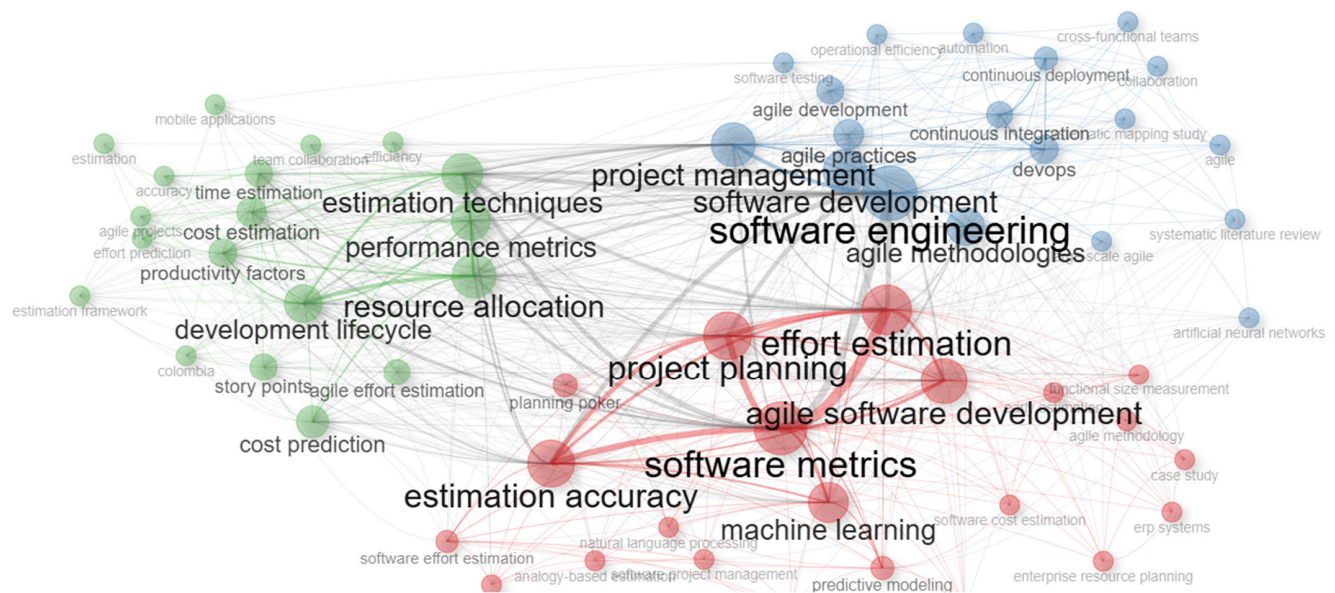


FIGURE 6. Keyword co-occurrence network map identifying thematic clusters within the literature.

sensitivity of such models to early sprint inaccuracies and unstable inputs.

D. COMMUNICATION AND TRANSPARENCY ISSUES

Although less frequently cited, communication gaps have high impact. Studies such as [S24] and [S38] emphasized that when stakeholders are not involved in estimation or when estimation assumptions are not clearly communicated, dissatisfaction rapidly escalates. Inadequate stakeholder involvement, especially in early sprint planning and backlog grooming sessions, creates mismatched expectations that ultimately lead to friction and rework. [S24] found that without cross-functional collaboration and clear communication about delivery expectations, DevOps initiatives often stall due to misunderstandings between development and operations teams regarding timelines and deliverables. A further complication arises when estimation logic, including key assumptions or constraints, is not made transparent. This opacity limits stakeholder trust and inhibits constructive feedback, as highlighted by [S63], who noted that misalignment between development and operations often stems from the lack of shared visibility into estimation rationales and status metrics, even in well-intentioned DevOps environments. Moreover, [S26] observed that fragmented communication across tools (e.g., GitLab, Slack, Jenkins) results in updates being dispersed and inconsistently relayed to relevant team members, which complicates real-time decision-making and disrupts sprint flow. Transparency is also critical when projects encounter delays. Yet studies report that teams often fail to explain root causes of timeline slippage in a timely or

coherent way. This worsens client frustration and diminishes credibility. [S41] argue that the rapid iterations in DevOps amplify this issue: without mechanisms to explain delivery fluctuations, even minor setbacks can erode client confidence.

Building on the issues identified in RQ1, this section explores the most effective approaches and techniques proposed in the literature to improve effort estimation accuracy and reliability in Agile and DevOps environments. Through a systematic review of relevant studies, several recurring strategies and innovations have emerged. These were categorized based on their nature, underlying methodology, and practical application context.

Rather than relying on a singular solution, the literature reveals a multifaceted ecosystem of effort estimation practices that address the technical, procedural, and human factors contributing to stakeholder dissatisfaction and estimation inaccuracy. To reflect this diversity, the identified techniques have been grouped into seven categories: (1) Machine Learning and Predictive Modelling, (2) Expert-Augmented Estimation, (3) Relative and Analogical Estimation Techniques, (4) Feedback-Oriented Learning Estimation, (5) Tool-Supported and Automated Estimation, and (6) Collaborative Estimation Practices, as summarized in Table 5. These categories provide a structured framework for researchers seeking to understand the challenges and influencing factors, as well as for practitioners aiming to apply actionable strategies that enhance accuracy, transparency, and stakeholder trust. The following sections examine each category in detail, highlighting their strengths, limitations, and relevance to Agile and DevOps-based projects.

TABLE 5. Descriptive overview of estimation approaches in the systematic review (N = 63).

Approaches	Studies	Limitations	Count	Percentage
Machine Learning and Predictive Modelling - Support Vector Regression (SVR) - Neural Networks - Non-linear Regression - Hybrid ML models	[S3][S4][S5][S59]	Requires large, clean historical data; limited interpretability in some models; setup and maintenance effort	4	6.03%
Expert-Augmented Estimation (Hybrid Models) - Hybrid Estimation Models - Planning Poker with Expert Oversight - Judgment-Calibrated ML	[S1][S3][S13]	Expert bias may affect results; requires regular recalibration; dependent on team experience	3	4.76%
Relative and Analogical Estimation Techniques - Story Points - T-shirt Sizing - Analogy-Based Estimation - Developer Velocity Models	[S1][S16][S54]	Needs consistent team understanding; risk of misalignment without retrospectives or calibration	3	4.76%
Tool-Supported and Automated Estimation - Automated Story Point Generators - Estimation Plugins (e.g., Jira ML) - CI/CD Analytics Tools	[S9][S23][S26][S27]	Requires integration into Agile tools; potential detachment from team ownership; over-reliance risks	4	6.03%
Collaborative Estimation Practices - Planning Poker - Wideband Delphi - Consensus Discussions - Cross-functional Estimation Workshops	[S1][S13]	Time-consuming for large teams; needs facilitation to avoid dominant voices; may trend toward compromise	2	3.17%
Feedback-Oriented Learning Estimation - Velocity-Based Training - Estimation Workshops - Sprint Retrospectives Estimation Accuracy Reviews - Feedback-Driven Learning	[S4][S16][S25][S51][S59][62]	Needs strong feedback loops, mentoring, learning and historical data; often sidelined in fast-paced environments	6	9.52%

E. MACHINE LEARNING AND PREDICTIVE MODELLING

Machine learning in the context of effort estimation is famous for fetching complexities and issues in the workflows of Agile Devops. These techniques use the history and past project data to develop modelling techniques which helps to predict development effort with lesser human bias yet greater contextual adaptability for the specific project [26], [27].

In the [S4] study, the authors introduced a maturity framework for machine learning based effort estimation, highlighting the importance of organizational preparedness and step-by-step integration of learning systems into agile planning. The framework supports continuous improvement by incorporating insights from previous sprints, which makes it suitable for iterative development processes. In the research [S5], the researchers made a multiple non-linear regression model for Agile projects. The model worked very well when there was enough past data and project details. This shows that machine learning models give better results when they have more and steady data to learn from.

The researchers [S3] tested a mix of Support Vector Regression and traditional estimation methods. This hybrid

approach gave more stable results than using only human estimates, especially in fast-changing projects with unclear or changing task lists. The [S59] study gives a big picture of how effort estimation tools have changed; from early neural networks to today's cloud-based machine learning tools. The study highlights that these new tools are helpful because they can grow easily and work automatically, which is very useful in large DevOps pipelines projects where doing manual estimates is difficult.

Even though machine learning methods are very powerful, they also have some problems. They usually need a lot of past data, clean and prepared input, and must be adjusted based on the project situation. Also, some machine learning models work like a "closed box," which means it is hard to understand how they make predictions. Because of this, team members or clients may not trust the results if there is no clear explanation [28].

F. EXPERT-AUGMENTED ESTIMATION (HYBRID MODELS)

Expert-augmented estimation is also called hybrid estimation which mixes formal models with human thinking. In Agile

and DevOps projects, team members often know the work well but may not have clear or steady data. This method gives a good balance between model-based estimates and human experience [13], [29]. In the [S3] study, expert opinions were added to a Support Vector Regression (SVR) model which helped to keep the expert's knowledge useful while also improving the model's accuracy with machine learning. This model helped reduced common human errors and made the model better at adjusting to changes in team speed and task complexity.

In the [S13] study, the researchers compared expert-based techniques with algorithm models in Agile projects. They found that expert opinions alone are not always consistent. But when experts input is combined with data-based models, the quality of estimates improves a lot and especially at the beginning of a project or when there is not much past data available.

The [S1] study compared different methods and showed the value of techniques like Planning Poker and T-shirt sizing. These methods use group sessions where team members from different roles share their ideas. When done correctly, they can make estimates more accurate. They also help team members better understand the user stories.

Expert-augmented models benefit from their adaptability and are especially effective in projects characterized by ambiguity or frequent change. However, their effectiveness hinges on the quality and experience of the involved experts and may require periodic calibration to avoid drift and overfitting to subjective judgments [30].

G. RELATIVE AND ANALOGICAL ESTIMATION TECHNIQUES

Relative and analogical estimation techniques are among the most widely used in Agile. These are favorite methods on basically all agile practices because they are very simple, adaptable to work within iterative workflows, and not time-bound as such. These techniques use comparisons between tasks based on their difficulty or past examples, making them easy for teams to understand when the backlog changes quickly [31].

One of the most prevalent approaches is story point estimation, which allows teams to assess user stories based on perceived effort, complexity, and risk. [S16] propose a model where story points are calibrated using individual developer velocity, enabling a more personalized and adaptive form of relative sizing. This approach enhances estimation consistency across sprints and supports better sprint planning.

[S1] compared common Agile estimation techniques, including story points and T-shirt sizing, and concluded that these methods offer practical advantages in team settings. They facilitate consensus-building and allow cross-functional teams to express effort in abstract units, which can be refined over time through retrospectives.

In addition, recent work [S54] investigated the correlation between story points and actual development time. Their

findings indicate that while not perfectly linear, story points remain a useful proxy for effort when consistently applied and reviewed. This reaffirms their value as a lightweight, low-overhead technique in Agile and DevOps teams.

However, the effectiveness of these methods heavily depends on team experience, calibration practices, and organizational consistency. When misused or applied without retrospection, they can lead to inflated estimates or misaligned expectations. Nevertheless, when properly implemented, relative estimation techniques enhance team autonomy and provide a scalable way to plan and track effort in dynamic environments.

H. FEEDBACK-ORIENTED LEARNING ESTIMATION

The human element specifically, the degree of expertise of the people making the estimations remains a crucial determinant of estimation accuracy, even though tools and techniques are significant. In order to achieve consistency and trust in Agile and DevOps planning processes, the literature is placing more and more emphasis on the development of estimation skills and organized training programs [32], [33].

[S4] introduced a maturity model that dynamically includes training and knowledge development as a necessary pillar in an organization's successful implementation of machine learning-enhanced estimation. Their framework suggests that organizations progress through levels of estimation maturity by investing in internal learning, including workshops, calibration exercises, and knowledge-sharing sessions.

[S16] proposed a training-based approach to effort estimation using specific velocity metrics for personalized use; this model can push individual developers to be able to use the estimation tool and learn on their own with the evaluation of their past results. It's not as if this increases the individual's predictions, but it may also serve to improve the overall consistency of the team.

[S59] emphasized the importance of education and continuous learning in the broader evolution of estimation practices, especially as teams transition from traditional models to more data-intensive or AI-supported techniques. The review advocates for the integration of estimation education into organizational onboarding and ongoing skill development programs.

Study [S62] suggested the best practices for process improvement, including calibration/tuning sessions which involves comparing estimated vs. the actual efforts in multiple iterations. These sorts of practices help teams improve their velocity metrics and standardize estimation baselines, reducing over time.

Despite its critical role, estimation training and feedback-oriented sessions are often under-prioritized due to time constraints or the assumption that it improves passively through experience. However, structured and intentional learning, especially when paired with retrospective feedback,

can significantly elevate team competence and reduce variability in estimates [34].

I. TOOL-SUPPORTED AND AUTOMATED ESTIMATION

Automation and tooling have become essential for effort estimation in Agile and DevOps workflows and are also expected to provide real-time insights, minimize the manual overhead, and increase the traceability. They would easily blend with the development pipelines to create or better effort estimates by drawing upon the backlogs, histories of sprints, and performance metrics [35].

[S9] made a sprint point estimation tool that automates the transformation of task attributes to story points. This approach saves time in planning and adds reliability to backlog management, a facility for distributed Agile teams, whose shared understanding is easily broken. [S23] came up with a model for integrating machine learning with estimation plugins in management tools like Jira Agile. The system learns from past estimates, developer performance, and delivery timeline, giving adaptive prediction which will evolve together with the team. This level of automation reduces cognitive load, allowing teams to deliver more and manually plan less.

[S27] evaluates project management tools that support sprint planning and tracking in a DevOps environment. They also emphasized that automation does not only mean the generation of estimates; it also means continuously updating and validating them as work progresses. It is high cycle time, particularly useful in high-frequency deployment environments.

Lastly, the study [S26] examined best practices and key DevOps tools for incorporating estimation functions into larger CI/CD pipelines. Their results demonstrate that tooling can greatly improve estimation accuracy, particularly when combined with deployment analytics, velocity trends, and performance metrics.

Automated tools offer scalability and efficiency, but they also add an abstraction layer that could make estimates less team-owned. Furthermore, if underlying assumptions change, an excessive reliance on automation without retrospective evaluation may result in the perpetuation of errors.

J. COLLABORATIVE ESTIMATION PRACTICES

Agile places a strong emphasis on collaboration, and estimation is no different. Planning Poker, Wideband Delphi, and team discussions are examples of consensus-driven techniques that are frequently used in collaborative estimation practices, which involve the collective input of team members to evaluate the effort required for tasks. These techniques lessen the effects of personal prejudice, increase openness, and foster mutual understanding [36].

After doing a comparative analysis of Agile estimate strategies, [S1] came to the conclusion that collaborative approaches such as Planning Poker not only provide higher estimation accuracy but also encourage increased team mem-

ber engagement and knowledge exchange. Because these methods are iterative, participants can test presumptions, improve their comprehension of the job, and eventually come to a more accurate estimate.

The effectiveness of expert-led team estimating techniques is further supported by [S13]. According to their findings, the process gains from a variety of viewpoints when several stakeholders participate in estimation, which produces forecasts that are more accurate and sensitive to context, particularly in challenging or innovative jobs.

Collaborative techniques are particularly useful in cross-functional Agile and DevOps teams, where roles such as developers, testers, and DevOps engineers contribute distinct insights. The open dialogue encouraged by these methods helps uncover hidden dependencies or overlooked technical challenges early in the planning cycle.

V. CONCLUSION

There are variety of approaches ranging from machine learning to expert judgement to collaborative estimations have been proposed to improve effort estimation in agile and devops environments, but no any universal solution exists. This shows a deeper reality, frustration with effort estimation isn't just about tools or methods, it's really a sign that something bigger is out of sync. This review has examined the reasons why effort estimation continues to be a challenge in Agile and DevOps environments, especially from the perspective of stakeholder satisfaction. The key issues identified fall into four main areas: human-related factors, technical limitations, gaps in estimation processes, and lack of clear communication. To address these challenges, the literature suggests a variety of approaches, such as using machine learning models, combining expert judgment with data, relying on analogies, improving through regular reviews and feedback, using estimation tools, working collaboratively, and training teams to improve estimation skills. The results show that no single method works in every situation. Instead, teams benefit most when they combine different methods based on their specific context, experience, and project needs. However, many studies still overlook the human and organizational side of estimation, such as team behaviour, company culture, and bias. To improve outcomes, teams should invest in training, use estimation tools effectively, and build a culture of learning through regular feedback. Future research should focus on evaluating the long-term effectiveness of various effort estimation approaches across diverse team structures and organizational culture using both numbers and real-world experiences to measure success.

APPENDIX PRIMARY STUDIES

This section provides the primary studies resulting from the selection process.

1. Mateen, M. A., & Malik, A. A. (2023). A comparative study of the accuracy and efficiency of wideband Delphi and planning poker software

- effort estimation techniques. In Proceedings of the 2023 International Conference on IT and Industrial Technologies (ICIT) (pp. 1–6). IEEE. <https://doi.org/10.1109/ICIT59216.2023.10335782>
2. Kaur, A., & Kaur, K. (2020). A COSMIC function points based test effort estimation model for mobile applications. *Journal of King Saud University - Computer and Information Sciences*, 32(3), 300–307. <https://doi.org/10.1016/j.jksuci.2019.03.001>.
 3. Bhaskaran, N. A., & Jayaraj, V. (2019). A hybrid effort estimation technique for agile software development (HEETAD). *International Journal of Engineering and Advanced Technology*, 9(1), 1078–1087. <https://doi.org/10.35940/ijeat.A9480.109119>.
 4. Mannan, M. A., Mustafa, S., & Hussain, A. (2024). A maturity level framework for practicing machine learning operations in CI/CD for software deployment. *Journal of Independent Studies and Research - Computing*, 22(1), 1–10. <https://doi.org/10.31645/JISRC.24.22.1.4>.
 5. Prykhodko, N. V., & Prykhodko, S. B. (2019). A multiple non-linear regression model to estimate the agile testing efforts for small web projects. *Radio Electronics, Computer Science, Control*, 2019(2), 158–166. <https://doi.org/10.15588/1607-3274-2019-2-17>.
 6. Khuat, T. T., & Le, M. H. (2018). A novel hybrid ABC-PSO algorithm for effort estimation of software projects using agile methodologies. *Journal of Intelligent Systems*, 27(3), 489–506. <https://doi.org/10.1515/jisys-2016-0294>.
 7. Rosa, W., & Jardine, S. (2024). A novel method to early agile effort estimation through functional initiatives. *Journal of Systems and Software*, 223, 112302. <https://doi.org/10.1016/j.jss.2024.112302>.
 8. Butt, S. A., Khalid, A., & Ercan, T. (2022). A software-based cost estimation technique in scrum using a developer's expertise. *Advances in Engineering Software*, 171, 103159. <https://doi.org/10.1016/j.advengsoft.2022.103159>.
 9. Popli, R., & Chauhan, N. (2019). A sprint point based tool for agile estimation. In M. Hoda, N. Chauhan, S. M. K. Quadri, & P. R. Srivastava (Eds.), *Software Engineering* (pp. 63–72). Springer. https://doi.org/10.1007/978-981-10-8848-3_6.
 10. Prakash, B. V., & Viswanathan, V. (2017). A survey on software estimation techniques in traditional and agile development models. *Indonesian Journal of Electrical Engineering and Computer Science*, 7(3), 867–876. <https://doi.org/10.11591/ijeecs.v7i3.pp867-876>.
 11. Jessica Ross (2024). Agile and DevOps: Elevating Software Quality through Collaborative Practices. *Journal of Science & Technology*. <https://thesciencebrigade.com/jst/article/view/50>
 12. Juan Cubillos and Jairo Aponte and Diana Gomez and Edwar Rojas (2024). Agile effort estimation in Colombia: An assessment and opportunities for improvement. *Science of Computer Programming*, 236(), 103115. <https://doi.org/10.1016/j.scico.2024.103115>
 13. Poženel, M., Fürst, L., Vavpotič, D., & Hovelja, T. (2023). Agile effort estimation: Comparing the accuracy and efficiency of Planning Poker, Bucket System, and Affinity Estimation methods. *International Journal of Software Engineering and Knowledge Engineering*, 33(11n12), 1–28. <https://doi.org/10.1142/S021819402350064X>.
 14. Tawosi, M., Moser, R., & Staron, M. (2024). Agile effort estimation: Have we solved the problem yet? Insights from the replication of the GPT2SP study. In *Proceedings of the 31st IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)* (pp. 1–11). IEEE. <https://doi.org/10.1109/SANER60148.2024.00111>.
 15. Meiliana, Daniella, G., & Wijaya, N. (2023). Agile software development effort estimation based on product backlog items. *Procedia Computer Science*, 227, 186–193. <https://doi.org/10.1016/j.procs.2023.10.516>.
 16. Rao, C. P., & Kumar, P. S. (2018). An agile effort estimation based on story points using machine learning techniques. In *Proceedings of the Second International Conference on Computational Intelligence and Informatics* (pp. 209–219). Springer. https://doi.org/10.1007/978-981-10-8228-3_20
 17. Raslan, A. T., & Darwish, N. R. (2018). An enhanced framework for effort estimation of agile projects. *International Journal of Intelligent Engineering and Systems*, 11(3), 205–214. <https://doi.org/10.22266/ijies2018.0630.22>.
 18. Malgonde, O., & Chari, K. (2019). An ensemble-based model for predicting agile software development effort. *Empirical Software Engineering*, 24(2), 1017–1055. <https://doi.org/10.1007/s10664-018-9647-0>.
 19. Srivastava, P., Srivastava, N., Agarwal, R., & Singh, P. (2023). An intelligent framework for estimating software development projects using machine learning. *International Journal on Recent and Innovation Trends in Computing and Communication*, 11(5), 160–169. <https://doi.org/10.17762/ijritcc.v11i5.6602>.
 20. Hameed, S., & Yousef, R. (2022). An optimized case-based software project effort estimation using genetic algorithm. *Information and Software Technology*, 145, 107088. <https://doi.org/10.1016/j.infsof.2022.107088>.
 21. Sousa, A. O., Veloso, D. T., Gonçalves, H. M., Faria, J. P., Mendes Moreira, J., Graça, R., Gomes, D., Castro, R. N., & Henriques, P. C. (2023). Applying machine learning to estimate the effort and duration of individual tasks in software projects. *IEEE Access*, 11, 89933–89946. <https://doi.org/10.1109/ACCESS.2023.3307310>.
 22. Thanh Tung Khuat and My Hanh Le (2017). Applying teaching-learning to artificial bee colony for parameter optimization of software effort estimation model. *Journal of Engineering Science and Technology*, 12(), 1178–1190.

23. Gupta, N., & Prasad, R. (2022). Automated software effort estimation for agile development system by heuristically improved hybrid learning. *Concurrency and Computation: Practice and Experience*, 34(17), e7267. <https://doi.org/10.1002/cpe.7267>.
24. Jayakody, J. A. V. M. K., & Wijayanayake, W. M. J. I. (2023). Critical success factors for DevOps adoption in information systems development. *International Journal of Information Systems and Project Management*, 11(3), 60–82. <https://doi.org/10.12821/ijispm110304>.
25. Alsaadi, B., & Saeedi, K. (2022). Data-driven effort estimation techniques of agile user stories: A systematic literature review. *Artificial Intelligence Review*, 55(7), 5485–5516. <https://doi.org/10.1007/s10462-021-10132-x>.
26. Eswararaj, D., Koppada, L. R., & Bodala, R. S. (2024). DevOps implementation: Essential tools, best practices, and solutions to overcome challenges for seamless development and operations integration. *Asian Journal of Research in Computer Science*, 17(10), 26–36. <https://doi.org/10.9734/ajrcos/2024/v17i10507>.
27. Angara, J., Prasad, S., & Sridevi, G. (2020). DevOps project management tools for sprint planning, estimation and execution maturity. *Cybernetics and Information Technologies*, 20(2), 79–92. <https://doi.org/10.2478/cait-2020-0018>.
28. Rivera Ibarra, J. G., Borrego, G., & Palacio, R. R. (2024). Early estimation in agile software development projects: A systematic mapping study. *Informatics*, 11(4), 81. <https://doi.org/10.3390/informatics11040081>.
29. Salmanoglu, M., Hacaloglu, T., & Demirors, O. (2017). Effort estimation for agile software development: Comparative case studies using COSMIC functional size measurement and story points. In *Proceedings of the 27th International Workshop on Software Measurement and the 12th International Conference on Software Process and Product Measurement* (pp. 41–49). ACM. <https://doi.org/10.1145/3143434.3143450>.
30. Kaur, A., & Kaur, K. (2019). Effort estimation for mobile applications using use case point (UCP). In *Smart Innovations in Communication and Computational Sciences* (pp. 163–172). Springer. https://doi.org/10.1007/978-981-10-8968-8_14.
31. Bilgaiyan, S., Mishra, S., & Das, M. (2019). Effort estimation in agile software development using experimental validation of neural network models. *International Journal of Information Technology*, 11(3), 569–573. <https://doi.org/10.1007/s41870-018-0131-2>.
32. Figueiredo, A. C., Pereira, R., & da Silva, M. Â. (2025). Exploring the integration of artificial intelligence and DevOps for agile product development. In *Digital Technologies and Transformation in Business, Industry and Organizations* (pp. 27–39). Springer. https://doi.org/10.1007/978-3-031-78412-5_2.
33. Sánchez Gordón, M., Colomo Palacios, R., & Kristiansen, M. (2022). *Effort estimation in agile software development: An exploratory study of practitioners' perspective*. In *Lecture Notes in Business Information Processing* (Vol. 536, pp. 136–149). Springer. https://doi.org/10.1007/978-3-030-94238-0_8.
34. Tanveer, B., Guzmán, L., & Engel, U.M. (2017). Effort estimation in agile software development: Case study and improvement framework. *Journal of Software: Evolution and Process*, 29(12), e1862. <https://doi.org/10.1002/smr.1862>.
35. Diana Gomez and Juan Cubillos and Jairo Aponte and Edwar Rojas (2023). Effort Estimation in Agile Software Development: The State of the Practice in Colombia. In *Anais do XXVI Congresso Ibero-Americano em Engenharia de Software (CIBSE 2023)* (pp. Jan-15). Sociedade Brasileira de Computação <https://doi.org/10.5753/cibse.2023.24689>.
36. Anureet Kaur and Kulwant Kaur (2018). Effort Estimation in Traditional and Agile Mobile Application Development & Testing. *Indonesian Journal of Electrical Engineering and Computer Science*, 12(), 1265. <http://doi.org/10.11591/ijeecs.v12.i3.pp1265-1272>
37. Yalçiner, B., Dinçer, K., Karaçor, A.G., & Efe, M. Ö. (2024). Enhancing agile story point estimation: Integrating deep learning, machine learning, and natural language processing with SBERT and gradient boosted trees. *Applied Sciences*, 14(16), 7305. <https://doi.org/10.3390/app14167305>.
38. Geetanjali Chakravorty, B. Ramachandra Reddy, & Danish Ali Khan (2025). *Enhancing effort and estimation in scrum based agile projects with a proposed federated agile framework*. *International Journal of Information Technology*, 17 (1), 1–15. <https://doi.org/10.1007/s41870-025-02451-x>.
39. Bashaer Alsaadi & Kawther Saeedi (2024). Ensemble effort estimation for novice agile teams. *Information and Software Technology*, 170, 107447. <https://doi.org/10.1016/j.infsof.2024.107447>.
40. Diwakar R. Tripathi (2021). Evaluation of the feasibility of parametric estimation in DevOps continuous planning. *International Journal for Research in Applied Science and Engineering Technology*, 9 (9), 542–546. <https://doi.org/10.22214/ijraset.2021.38008>.
41. Fernando Almeida, Jorge Simões, & Sérgio Lopes (2022). *Exploring the benefits of combining DevOps and Agile*. *Future Internet*, 14 (2), 63. <https://doi.org/10.3390/fi14020063>.
42. K. Eswara Rao, Pandu Ranga Vital Terlapu, Paidi Annan Naidu, Tammineni Ravi Kumar, & Bala Murali Pydi (2024). *Feature importance for software development effort estimation using multi level ensemble approaches*. *Bulletin of Electrical Engineering and Informatics*, 13 (2), 1090–1102. <https://doi.org/10.11591/eei.v13i2.5531>.
43. Dulani Meedeniya & Harshanika Thennakoon (2021). *Impact factors and best practices to improve effort*

- estimation strategies and practices in DevOps. In *Proceedings of the 11th International Conference on Information Communication and Management* (pp. 11–17). ACM. <https://doi.org/10.1145/3484399.3484401>.
44. Shahida Hayat, Wajahat Akbar, Tariq Hussain, Muhammad Inam Ul Haq, Altaf Hussian, Irshad Khalil, Muhammad Nawaz Khan & Samsonova Diana (2024). *Improving effort estimation accuracy in software development projects using multiple imputation techniques for missing data handling*. *IECE Transactions on Intelligent Systematics*, 1 (3), 190–202. <https://doi.org/10.62762/TIS.2024.751418>.
 45. Arun Pandiyan Perumal & Pradeep Chintale (2022). *Improving operational efficiency and productivity through the fusion of DevOps and SRE practices in multi cloud operations*. *International Journal of Cloud Computing and Database Management*, 3 (2), 49–53. <https://doi.org/10.33545/27075907.2022.v3.i2a.5>.
 46. Ikeoluwa Kolawole & Akinwumi Fakokunde (2024). *Improving software development with continuous integration and deployment for agile DevOps in engineering practices*. *International Journal of Computer Applications Technology and Research*, 14 (1), 25–39. <https://doi.org/10.7753/IJCATR1401.1002>
 47. Joshua Idowu Akerere, Abel Uzoka, Pascal Ugochukwu Ojukwu & Olugbenga Jeremiah Olamijuwon (2024). *Increasing software deployment speed in agile environments through automated configuration management*. *International Journal of Engineering Research Updates*, 7 (2), 028–035. <https://doi.org/10.20894/IJERU-2024-0047>.
 48. Perna Singal, Prabha Sharma & A. Charan Kumari (2022). *Integrating software effort estimation with risk management*. *International Journal of System Assurance Engineering and Management*, 13 (5), 2413–2428. <https://doi.org/10.1007/s13198-022-01652-y>.
 49. Franziska Tobisch, Karla Weigelt, Pascal Philipp & Florian Matthes (2024). *Investigating effort estimation in a large scale agile ERP transformation program*. In D. Smite, E. Guerra, X. Wang, M. Marchesi & P. Gregory (Eds.), *Agile Processes in Software Engineering and Extreme Programming – 25th International Conference on Agile Software Development, XP 2024, Proceedings* (Lecture Notes in Business Information Processing, Vol. 512, pp. 70–86). Springer. https://doi.org/10.1007/978-3-031-61154-4_5.
 50. Amrita Sharma & Neha Chaudhary (2020). *Linear regression model for agile software development effort estimation*. In *2020 5th IEEE International Conference on Recent Advances and Innovations in Engineering (ICRAIE)* (pp. ...). IEEE. <https://doi.org/10.1109/ICRAIE51050.2020.9358309>.
 51. Renato Menezes, Marcelo Marinho & Suzana Sampaio (2024). *Metrics in large scale agile software development: A multivocal literature review*. In *Anais do XXVII Congresso Ibero Americano em Engenharia de Software (CIBSE 2024)* (pp. 106–120). Sociedade Brasileira de Computação. <https://doi.org/10.5753/cibse.2024.28442>.
 52. Radek Silhavy, Miroslav Bures, Melchizedek Alipio & Petr Silhavy (2023). *More accurate cost estimation for Internet of Things projects by adaptation of use case points methodology*. *IEEE Internet of Things Journal*, 10 (21), 19312–19327. <https://doi.org/10.1109/JIOT.2023.3281614>.
 53. Shailesh Kumar (2019). *Normalized sprint estimation*. *International Journal of Productivity Management and Assessment Technologies*, 7 (2), 41–58. <https://doi.org/10.4018/IJPMAT.2019070103>.
 54. Vali Tawosi, Rebecca Moussa & Federica Sarro (2022). *On the relationship between story points and development effort in agile open source software*. In *Proceedings of the 16th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM 2022)* (pp. 183–194). ACM. <https://doi.org/10.1145/3544902.3546238>.
 55. Ibtissam Abnane, Ali Idri & Alain Abran (2024). *Optimized fuzzy clustering-based k-nearest neighbors imputation for mixed missing data in software development effort estimation*. *Journal of Software: Evolution and Process*, 36(3), e2586. <https://doi.org/10.1002/smr.2529>.
 56. Prisca Amajuoyi, Lucky Bamidele Benjamin & Kudirat Bukola Adeusi (2024). *Optimizing agile project management methodologies in high-tech software development*. *GSC Advanced Research and Reviews*, 19(1), 268–274. <https://doi.org/10.30574/gscarr.2024.19.2.0182>.
 57. Rismanda Kusumadewi & Ronald Adrian (2023). *Performance analysis of DevOps practice implementation of CI/CD using Jenkins*. *MATICS: Jurnal Ilmu Komputer dan Teknologi Informasi*, 15(2), 90–95. <https://doi.org/10.18860/mat.v15i2.17091>.
 58. Shariq Aziz Butt, Tuncay Ercan, Muhammad Binsawad, Paola-Patricia Ariza-Colpas, Jorge Diaz-Martinez, Gabriel Piñeres-Espitia, Emiro De-La-Hoz-Franco, Marlon Alberto Pineres Melo, Roberto Morales Ortega & Juan-David De-La-Hoz-Hernández (2023). *Prediction-based cost estimation technique in agile development*. *Advances in Engineering Software*, 175, 103408. <https://doi.org/10.1016/j.advengsoft.2022.103329>.
 59. Rijwani, P., & Jain, S. (2022). *Software effort estimation development from neural networks to deep learning approaches*. *Journal of Cases on Information Technology*, 24(4), 1–16. <https://doi.org/10.4018/JCIT.296715>.
 60. Sakhrawi, Z., Sellami, A., & Bouassida, N. (2022). *Software enhancement effort estimation using correlation-based feature selection and stacking ensemble method*. *Cluster Computing*, 25, 2779–2792. <https://doi.org/10.1007/s10586-021-03447-5>.

61. Saeed, A., Butt, W. H., Kazmi, F., & Arif, M. (2018). Survey of software development effort estimation techniques. In *Proceedings of the 2018 7th International Conference on Software and Computer Applications* (pp. 82–86). ACM. <https://doi.org/10.1145/3185089.3185140>
 62. Brian Pando and Abraham Dávila (2023). Systematic literature review on improving software requirements engineering processes. *Proceedings of the Institute for System Programming of the RAS*, 35(), 163–188.
 63. Wiedemann, A., Wiesche, M., Gewald, H., & Krcmar, H. (2020). Understanding how DevOps aligns development and operations: A tripartite model of intra-IT alignment. *European Journal of Information Systems*, 29(5), 458–473. <https://doi.org/10.1080/0960085X.2020.1782277>.
- ## REFERENCES
- [1] K. Zavrazhnyi and A. Kulyk, “Comparative characteristics of the managerial and economic aspects of digital business transformation,” *Econ. Syst. Develop.*, vol. 5, no. 2, pp. 27–32, Nov. 2023, doi: [10.32782/2707-8019/2023-2-3](https://doi.org/10.32782/2707-8019/2023-2-3).
 - [2] C. K. Kotler, “Enhancing U.S. software companies’ competitive edge with agile project management,” *J. Entrepreneurship Project Manage.*, vol. 7, no. 10, pp. 1–13, Oct. 2023, doi: [10.53819/810181024197](https://doi.org/10.53819/810181024197).
 - [3] N. A. Zakaria, A. R. Ismail, A. Y. Ali, N. H. M. Khalid, and N. Z. Abidin. (2021). *Software Project Estimation With Machine Learning*. [Online]. Available: <http://promise.site.uottawa.ca/serepository/datasets-page.html>
 - [4] A. M. Salih, S. M. Syed-Mohamad, P. Keikhosrokiani, and N. H. Samsudin, “Adopting DevOps practices: An enhanced unified theory of acceptance and use of technology framework,” *Int. J. Electr. Comput. Eng. (IJECE)*, vol. 13, no. 6, p. 6701, Dec. 2023, doi: [10.11591/ijece.v13i6.pp6701-6717](https://doi.org/10.11591/ijece.v13i6.pp6701-6717).
 - [5] M. Turic, S. Celar, S. Dragicevic, and L. Vickovic, “Advanced Bayesian network for task effort estimation in agile software development,” *Appl. Sci.*, vol. 13, no. 16, p. 9465, Aug. 2023, doi: [10.3390/app13169465](https://doi.org/10.3390/app13169465).
 - [6] S. A. Ruk, M. F. Khan, S. G. Khan, and S. M. Zia, “A survey on adopting agile software development: Issues & its impact on software quality,” in *Proc. IEEE 6th Int. Conf. Eng. Technol. Appl. Sci. (ICETAS)*, Dec. 2019, pp. 1–5, doi: [10.1109/ICETAS48360.2019.9117324](https://doi.org/10.1109/ICETAS48360.2019.9117324).
 - [7] C.-E. Orozco-Garcés, C.-J. Pardo-Calvache, and Y.-H. Salazar-Mondragón, “What is there about DevOps assessment? A systematic mapping,” *Revista Facultad de Ingeniería*, vol. 31, no. 59, Mar. 2022, Art. no. e13896, doi: [10.19053/01211129.v31.n59.2022.13896](https://doi.org/10.19053/01211129.v31.n59.2022.13896).
 - [8] L. B. Angarita, A. F. D. Carpio, and A. A. Osorio Londoño, “A bibliometric analysis of DevOps metrics,” *DESIDOC J. Library Inf. Technol.*, vol. 42, no. 6, pp. 387–396, Jan. 2023, doi: [10.14429/djlit.42.6.18365](https://doi.org/10.14429/djlit.42.6.18365).
 - [9] A. Wiedemann, M. Wiesche, H. Gewald, and H. Krcmar, “Understanding how DevOps aligns development and operations: A tripartite model of intra-IT alignment,” *Eur. J. Inf. Syst.*, vol. 29, no. 5, pp. 458–473, Sep. 2020, doi: [10.1080/0960085X.2020.1782277](https://doi.org/10.1080/0960085X.2020.1782277).
 - [10] M. Gall and F. Pigni, “Taking DevOps mainstream: A critical review and conceptual framework,” *Eur. J. Inf. Syst.*, vol. 31, no. 5, pp. 548–567, Sep. 2022, doi: [10.1080/0960085X.2021.1997100](https://doi.org/10.1080/0960085X.2021.1997100).
 - [11] J. Pasuksmi, P. Thongtanunam, and S. Karunasekera, “A systematic literature review on reasons and approaches for accurate effort estimations in agile,” *ACM Comput. Surveys*, vol. 56, no. 11, pp. 1–37, Nov. 2024, doi: [10.1145/3666365](https://doi.org/10.1145/3666365).
 - [12] D. Gomez, J. Cubillos, J. Aponte, and E. Rojas, “Effort estimation in agile software development: The state of the practice in Colombia,” in *Proc. 28th Anais do Congresso Ibero-Americano em Engenharia de Softw.*, Apr. 2023, pp. 1–15, doi: [10.5753/cibse.2023.24689](https://doi.org/10.5753/cibse.2023.24689).
 - [13] Y. Mahmood, N. Kama, and A. Azmi, “A systematic review of studies on use case points and expert-based estimation of software development effort,” *J. Softw., Evol. Process*, vol. 32, no. 7, p. e2245, Jul. 2020, doi: [10.1002/smr.2245](https://doi.org/10.1002/smr.2245).
 - [14] B. Alsaadi and K. Saeedi, “Ensemble effort estimation for novice agile teams,” *Inf. Softw. Technol.*, vol. 170, Jun. 2024, Art. no. 107447, doi: [10.1016/j.infsof.2024.107447](https://doi.org/10.1016/j.infsof.2024.107447).
 - [15] T. J. Gandomani, M. Dashti, H. Zulzalil, and A. B. M. Sultan, “Enhancing software effort estimation in the analogy-based approach through the combination of regression methods,” *IEEE Access*, vol. 12, pp. 152122–152137, 2024, doi: [10.1109/ACCESS.2024.3480829](https://doi.org/10.1109/ACCESS.2024.3480829).
 - [16] IFPUG. (2023). *Metric Views 'rethinking Software Development Estimates'*. Accessed: Sep. 16, 2024. [Online]. Available: <https://www.pageturnpro.com/Creative-Marketing-Alliance/110223-IFPUG-November-2023-MetricViews/sdefault.html#page/1>
 - [17] J. Ali, “Impact engineering: Transforming beyond agile project management,” Engprax Ltd., 2024. [Online]. Available: <https://www.amazon.co.uk/dp/106860574X>
 - [18] H. P. A. John. (2009). *10 Deploys Per Day Dev & Ops Cooperation at Flickr*. Accessed: Feb. 26, 2025. [Online]. Available: <https://paulhammond.org/2009/06/velocity/10deploys.pdf>
 - [19] D. Meedeniya and H. Thennakoon, “Impact factors and best practices to improve effort estimation strategies and practices in DevOps,” in *Proc. 11th Int. Conf. Inf. Commun. Manage.*, Aug. 2021, pp. 11–17, doi: [10.1145/3484399.3484401](https://doi.org/10.1145/3484399.3484401).
 - [20] J. Angara, S. Prasad, and G. Sridevi, “DevOps project management tools for sprint planning, estimation and execution maturity,” *Cybern. Inf. Technol.*, vol. 20, no. 2, pp. 79–92, Jun. 2020, doi: [10.2478/cait-2020-0018](https://doi.org/10.2478/cait-2020-0018).
 - [21] A. Carrera-Rivera, W. Ochoa, F. Larrinaga, and G. Lasar, “How-to conduct a systematic literature review: A quick guide for computer science research,” *MethodsX*, vol. 9, Jul. 2022, Art. no. 101895, doi: [10.1016/j.mex.2022.101895](https://doi.org/10.1016/j.mex.2022.101895).
 - [22] B. Kitchenham, L. Madeyski, and D. Budgen, “SEGRESS: Software engineering guidelines for RReporting secondary studies,” *IEEE Trans. Softw. Eng.*, vol. 49, no. 3, pp. 1273–1298, Mar. 2023, doi: [10.1109/TSE.2022.3174092](https://doi.org/10.1109/TSE.2022.3174092).
 - [23] B. Kitchenham and S. Charters, “Guidelines for performing systematic literature reviews in software engineering,” Keele Univ., Univ. Durham, U.K., Tech. Rep. EBSE-2007-01, 2007.
 - [24] S. Hasanoff, D. Pollock, T. H. Barker, and Z. Munn, “Tools to assess the risk of bias of evidence syntheses: A scoping review protocol,” *JBI Evidence Synth.*, vol. 22, no. 3, pp. 472–480, Mar. 2024, doi: [10.1112/jbies-23-00316](https://doi.org/10.1112/jbies-23-00316).
 - [25] J. Pérez, J. Díaz, J. Garcia-Martin, and B. Tabuenca, “Systematic literature reviews in software engineering—Enhancement of the study selection process using Cohen’s Kappa statistic,” *J. Syst. Softw.*, vol. 168, Oct. 2020, Art. no. 110657, doi: [10.1016/j.jss.2020.110657](https://doi.org/10.1016/j.jss.2020.110657).
 - [26] H. D. P. Carvalho, M. N. C. A. Lima, W. B. Santos, and R. A. de A. Fagunde, “Ensemble regression models for software development effort estimation: A comparative study,” *Int. J. Softw. Eng. Appl.*, vol. 11, no. 3, pp. 71–86, May 2020, doi: [10.5121/ijsea.2020.11305](https://doi.org/10.5121/ijsea.2020.11305).
 - [27] P. V. Terlapu, K. K. Raju, G. Kiran Kumar, G. Jagadeeswara Rao, K. Kavitha, and S. Samreen, “Improved software effort estimation through machine learning: Challenges, applications, and feature importance analysis,” *IEEE Access*, vol. 12, pp. 138663–138701, 2024, doi: [10.1109/ACCESS.2024.3457771](https://doi.org/10.1109/ACCESS.2024.3457771).
 - [28] K. Lavingia, R. Patel, V. Patel, and A. Lavingia, “Software effort estimation using machine learning algorithms,” *Scalable Computing: Pract. Exper.*, vol. 25, no. 2, pp. 1276–1285, Feb. 2024, doi: [10.12694/scpe.v25i2.2213](https://doi.org/10.12694/scpe.v25i2.2213).
 - [29] V. Teslyuk, A. Batyuk, and V. Voityshyn, “Preliminary estimation for software development projects empowered with a method of recommending optimal duration and team composition,” *Appl. Syst. Innov.*, vol. 7, no. 3, p. 34, Apr. 2024, doi: [10.3390/asi7030034](https://doi.org/10.3390/asi7030034).
 - [30] J. G. Rivera Ibarra, G. Borrego, and R. R. Palacio, “Early estimation in agile software development projects: A systematic mapping study,” *Informatics*, vol. 11, no. 4, p. 81, Nov. 2024, doi: [10.3390/informatics11040081](https://doi.org/10.3390/informatics11040081).
 - [31] M. Latif, M. Khan, and U. Hani, “Using standard deviation with analogy-based estimation for improved software effort prediction,” *KSII Trans. Internet Inf. Syst.*, vol. 17, no. 5, pp. 1356–1376, May 2023, doi: [10.3837/tiis.2023.05.003](https://doi.org/10.3837/tiis.2023.05.003).
 - [32] E. Feizpour, H. Tahayori, and A. Sami, “CoBRA without experts: New paradigm for software development effort estimation using COCOMO metrics,” *J. Softw., Evol. Process*, vol. 35, no. 12, pp. 4–8, Dec. 2023, doi: [10.1002/smr.2569](https://doi.org/10.1002/smr.2569).
 - [33] J. Zulkarnain, R. F. Mulya, T. Pratiwi, W. Pangestuti, and F. I. Ilmawati, “DevOps main area and core capabilities adopting DevOps in the last decade: A systematic literature review,” *Int. J. Res. Appl. Technol.*, vol. 2, no. 2, pp. 184–197, Dec. 2022, doi: [10.34010/injuratech.v2i2.8364](https://doi.org/10.34010/injuratech.v2i2.8364).

- [34] K. S. Thant and H. H. K. Tin, "Learning the efficient estimation techniques for successful software project management," *Innovare J. Eng. Technol.*, vol. 11, no. 6, pp. 4–8, May 2023, doi: [10.22159/ijet.2023.v11i1.47605](https://doi.org/10.22159/ijet.2023.v11i1.47605).
- [35] D. Nyale, "Unravelling DevOps agile methodologies: A comprehensive review of recent research," *Int. J. for Res. Appl. Sci. Eng. Technol.*, vol. 11, no. 11, pp. 2012–2021, Nov. 2023, doi: [10.22214/ijraset.2023.56986](https://doi.org/10.22214/ijraset.2023.56986).
- [36] D. J. Ross, "Agile and DevOps: Elevating software quality through collaborative practices," *J. Sci. Technol.*, vol. 5, no. 1, pp. 38–55, Jan. 2024.



SADAQUAT ALI RUK received the B.E. and M.E. degrees in software engineering from the Mehran University of Engineering and Technology, Jamshoro, Pakistan, in 2014 and 2018, respectively. He is currently pursuing the Ph.D. degree with the Faculty of Artificial Intelligence, Universiti Teknologi Malaysia. He is an Assistant Professor with the Department of Computer Science, Shah Abdul Latif University, Khairpur, Ghotki Campus, Pakistan. His research interests include agile software effort estimation, ensemble modeling techniques for DevOps, machine learning applications in energy management, the Internet of Things (IoT), natural language processing, and database performance optimization. To date, he has authored over fifteen peer-reviewed articles in IEEE- and Scopus-indexed venues.



YASIR MAHMOOD received the Ph.D. degree in software engineering from Universiti Teknologi Malaysia (UTM), Kuala Lumpur, Malaysia, in 2022. He is currently an Assistant Professor with the Department of Computer Science and Software Engineering, College of Information Technology, United Arab Emirates University (UAEU). He also holds the role of course coordinator for both undergraduate and postgraduate programs within the department. With over 17 years of teaching and administrative experience in national and international higher education institutions, he brings a strong academic and leadership background to his role. He has authored more than 15 research articles in reputable academic journals and is the author of the book titled *Software Requirement Change Effort Estimation: An Algorithmic Approach*. His research interests include software engineering, software effort and cost estimation, machine learning, requirements engineering, human–computer interaction, software design, software testing, and software maintenance.



AZRI AZMI received the bachelor's degree in computer science (software engineering), the master's degree in real-time software engineering, and the Ph.D. degree in computer science from Universiti Teknologi Malaysia (UTM), Kuala Lumpur. He is currently an Associate Professor at the Faculty of Artificial Intelligence, UTM. Over more than 25 years in academia, he has held numerous leadership and academic roles, including the Director of the Advanced Informatics Department and the Coordinator of the Master of Software Engineering Program. His research interests include software engineering, software architecture and design, software testing, and effort estimation models, particularly within Agile and DevOps environments. His current research interests include intelligent systems and secure software engineering, such as the development of deep learning- and machine learning-based techniques for detecting Android vulnerabilities. He has led and contributed to a wide range of national research grants funded by the Ministry of Education, with a strong publication record in indexed journals and international conferences. He has been honored with several academic awards, including an Excellent Supervisor recognitions and Top Cited Article accolades in Web of Science-indexed journals. He is also a certified tester by the International Software Testing Qualifications Board (ISTQB) and a registered Professional Technologist with Malaysia Board of Technologists (MBOT).



NURULHUDA FIRDAUS MOHD AZMI received the B.Sc. degree in computer science from Universiti Teknologi Malaysia (UTM), the M.Sc. degree in applied statistics from Universiti Putra Malaysia (UPM), and the Ph.D. degree in computer science from the University of York, U.K., where her research focused on artificial immune systems for adaptive information filtering. She is currently an Associate Professor at the Faculty of Artificial Intelligence, UTM. She is actively engaged in research in the fields of artificial intelligence and data analytics, with interests in intelligent systems, adaptive filtering, machine learning, and explainable AI. She has authored and co-authored numerous publications in reputable journals and conferences. Her work has contributed to various applications in healthcare, education, and decision support systems.

Dr. Azmi is a member of Malaysia Association of Information Systems (MyAIS) and the Association for Computing Machinery (ACM), reflecting her commitment to both national and international research communities. She regularly serves as a reviewer and a committee member for AI-related academic platforms.



SEHAR GUL was born in Sukkur, Sindh, Pakistan, in 1992. She received the B.E. degree in software engineering and the M.E. degree in software engineering from the Mehran University of Engineering and Technology (MUET), Jamshoro, in 2014 and 2018, respectively. She is currently pursuing the Ph.D. degree with the Faculty of Computing, Universiti Teknologi Malaysia (UTM), Kuala Lumpur, Malaysia. From 2014 to 2019, she was the Team Lead and a Software Developer at Sapphire Consulting Services, where she led various software development initiatives and contributed to enterprise-level projects. In 2021, she was awarded a prestigious HEC-funded scholarship by the Government of Pakistan to pursue her Ph.D. studies abroad. Her earlier research work involved sentiment analysis and agile software development, while her current research focuses on addressing class imbalance problems in machine learning classification tasks. She has authored eight research articles in diverse areas of computer science and software engineering. Her research interests include machine learning, data imbalance handling, sentiment analysis, agile methodologies, and intelligent systems.

...